JavaScript

The Language of the Web

Computer Science — 1st Year Presentation

Group Members

Boualem Bayoud | Aniss Kara

Abdesselam Ahmed Amine | Abd El Djalil Adjou

University of Bouira | April 6, 2026

Table of Contents

CHAPTER 1

Introduction

What is JS ▪ History ▪ The Web Trio ▪ Key Facts

University of Bouira | April 6, 2026

```
    return "Hello, World!";
}
```

JavaScript is a `high-level`, `interpreted` and `dynamic` programming language — the **only** language that runs natively inside every web browser.

Front-End

Makes web pages **interactive**, animated and responsive — without reloading from the server.

Back-End

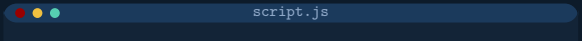
Runs on servers via **Node.js** to handle databases, APIs and real-time applications.

Mobile & Desktop

Cross-platform apps via **React Native** (mobile) and **Electron** (desktop).

Key Facts

- Used by **98%** of all websites worldwide
- Runs in every modern browser — *no installation needed*
- Part of the essential **HTML / CSS / JS** trio
- **#1** most used language on GitHub for years
- Huge ecosystem: **React, Vue, Angular, Node.js**



1995 — **Brendan Eich** creates **Mocha** (later renamed **JavaScript**) at *Netscape* in just **10 days**.

1996 — Microsoft releases JScript for Internet Explorer — the "browser wars" begin.

1997 — Standardised as **ECMAScript** by ECMA International — one specification for all browsers.

2009 — **Node.js** released — JavaScript now runs on *servers*, not just browsers.

Now — **#1** language on GitHub. Powering the entire modern web.

hyperref

Creator of JavaScript

Brendan Eich

- Born: July 4, 1961
- American programmer
- Co-founder of Mozilla
- Created JavaScript in **May 1995**
- Built in only **10 days!**

Why was JavaScript created?

Netscape needed a **lightweight scripting language** so that web designers — not just programmers — could make pages **interactive** without reloading them from the server every time.

Fields of Application › Where JavaScript is used in the real world

Web Development

Powers **every** interactive website. Gmail, YouTube, Twitter — all built with JS.

Mobile Apps

React Native builds iOS & Android apps from a single JavaScript codebase.

Games & AI

Browser games via Canvas API. ML models via **TensorFlow.js** in the browser.

More fields

- **Desktop apps** — Electron (VS Code, Slack, Discord)
- **Server & APIs** — Node.js + Express (Netflix, Uber, LinkedIn)
- **Real-time apps** — WebSockets, Socket.io (live chat, collaboration)
- **IoT & Robotics** — Johnny-Five (hardware programming with JS)
- **Browser extensions** — Chrome & Firefox extensions are written in JS

By the Numbers

- **12 years in a row** — most used language
- **#1 on GitHub**
- **98%** of websites use JS
- **2+ million** npm packages
- **16+ million** developers

Top Languages — GitHub 2024
Source: GitHub Octoverse 2024

Career Opportunities & Salaries › What knowing JavaScript can get you

Front-End Developer

Builds user interfaces. Avg. salary: **$75k–$110k/yr** (USA)

Back-End Developer

Builds APIs & servers with Node.js. Avg. salary: **$85k–$120k/yr** (USA)

Full-Stack Developer

Masters both front & back end. Avg. salary: **$95k–$140k/yr** (USA)

Mobile Developer

Builds apps with React Native. Avg. salary: **$80k–$115k/yr** (USA)

Job Market Highlights

- JavaScript skills appear in **over 70%** of all web developer job postings worldwide
- **Remote-friendly** — most JS jobs can be done fully remotely
- Gateway to **TypeScript**, increasingly required in enterprise positions
- Strong demand in **startups**, tech companies, banks and government agencies

Algeria & North Africa

Growing demand for JS developers in local startups, freelance & remote work.

CHAPTER 2

JS Basics

Variables ▪ Data Types ▪ Operators ▪ Arrays ▪ Objects

University of Bouira | April 6, 2026

Chapter Overview › What we will cover in the next ~3 minutes

Variables

Storing data

with `let` & `const`

Data Types

string, number,

boolean & more

Operators

Arithmetic,

comparison & logic

Why does this matter?

Variables, types and operators are the **building blocks** of every program. Before you can write logic, you need to know how to **store** and **work with** data.

- A **variable** is a named container for storing a value
- `let` — value **can** change later
- `const` — value **cannot** change once set
- Always prefer `const` by default; use `let` only when you need to reassign

```
// let — can be reassigned
let score = 0;
score = 10; // ✓ works
// const — fixed value
const name = "Djalil";
// name = "Other"; ✗ error
// Use const for fixed data
const PI = 3.14159;
let count = 0;
count++; // count is now 1
```

Avoid `var`

The old keyword `var` has confusing scoping rules. Modern JS uses `let` and `const` instead.

Data Types › The kinds of values JavaScript can store

A string

Text wrapped in quotes. `"Hello"`, `"JavaScript"`. Used for names, messages, any text.

number

Any numeric value: integers or decimals. `42`, `3.14`, `-7`. Used for scores, prices, counts.

boolean

Only two values: `true` or `false`. Used in conditions and logic checks.

Other types to know

- `null` / `undefined` — empty values
- `array` — ordered list: `[1, 2, 3]`
- `object` — key-value pairs
- `Error` — **Exception Handling**:
  `try { ... } catch (e) { ... }`

Pro tip

Use `typeof x` to check
the type of any variable at runtime.

+ Arithmetic Operators

code-slide5

```
let a = 10, b = 3;
a + b   // 13 — addition
a - b   // 7  — subtraction
a * b   // 30 — multiply
a / b   // 3.33 — divide
a % b   // 1  — remainder
```

ℹ % gives the **remainder** of a division.

Example: `10 % 3 = 1`

= Comparison & Logical

code-slide5

```
5 == "5"   // true (loose)
5 === "5"  // false (strict)
10 > 3     // true
2 !== 5    // true
```

⚠ **Always use ===**

`===` checks value **and** type.

`==` checks value only — avoid it.

- Declare data with `const` / `let`
- Give it the right **type** (string, number, boolean . . .)
- Use **operators** to compute, compare and combine values
- The result feeds directly into **conditions** and **functions**

code-slide6

```
1  // Variables
2  const studentName = "Djalil";
3  let score = 85;
4  const passed = score >= 50;
5  // Operators
6  let bonus = score > 80 && passed;
7  // Output
8  console.log(
9    studentName + " passed: " + passed
10 );
11 // → "Djalil passed: true"
```

→ What comes next

Chapter 2 will use everything you just learned inside:

- `if` / `else` conditions

CHAPTER 3

Control & Logic

Conditions ▪ Loops ▪ Functions ▪ Frameworks

University of Bouira | April 6, 2026

```
        return "Hello, World!";
    }
```

Chapter Overview › What we will cover in the next ~6 minutes

Conditions

Deciding what code runs

Loops

Repeating actions

Functions

Reusable blocks of code

Frameworks

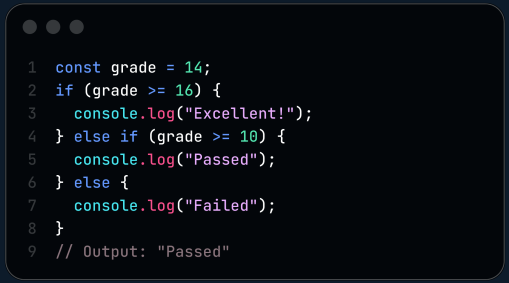
React & Node.js

Why does this matter?

Control structures are the **skeleton** of every program. Without them, code would just run line-by-line with no decisions, no repetition, and no reuse.

- Code runs **only when** a condition is `true`
- `else if` chains multiple conditions
- `else` is the final fallback
- Conditions use **comparison operators**:

  `===` `!==` `>` `<` `>=` `<=`



```
const grade = 14;
if (grade >= 16) {
  console.log("Excellent!");
} else if (grade >= 10) {
  console.log("Passed");
} else {
  console.log("Failed");
}
// Output: "Passed"
```

Remember

Always use `===` (strict equality), not `==` (loose equality).

Conditions — Ternary & Switch › Shorter syntax for simple choices

Ternary Operator

```
// condition ? valueA : valueB
const age = 20;
const status = age >= 18
  ? "Adult"
  : "Minor";
console.log(status); // "Adult"
```

Best for **simple, one-line** decisions.
Avoid nesting ternaries — hard to read.

Switch Statement

```
const day = "Monday";
switch (day) {
  case "Monday":
    console.log("Week starts!");
    break;
  case "Friday":
    console.log("Weekend soon!");
    break;
  default:
    console.log("Regular day");
}
```

`break` stops fall-through.

A `for` loop has **3 parts**:

- **Initializer** — `let i = 0`
  Runs once at the start
- **Condition** — `i < 5`
  Checked before each iteration
- **Update** — `i++`
  Runs after each iteration

Output

```
1 2 3 4 5
```

```
// Count from 1 to 5
for (let i = 1; i <= 5; i++) {
  console.log(i);
}
// Loop over an array
const students = [
  "Ali", "Sara", "Youssef"
];
for (let i = 0;
     i < students.length; i++) {
  console.log(students[i]);
}
```

Loops — `while` & `for...of` › Two more ways to repeat code

while — Unknown iterations

```
let count = 0;

while (count < 3) {
  console.log("Count: " + count);
  count++;
}
// Count: 0
// Count: 1
// Count: 2
```

⚠ Always update the variable inside the loop or it runs forever!

for...of — Clean array iteration

```
const fruits = [
  "apple", "mango", "fig"
];

for (const fruit of fruits) {
  console.log(fruit);
}
// apple
// mango
// fig
```

✓ Cleaner than `for` when you don't need the index. Preferred modern JS.

Functions — Reusable Blocks of Code › Write once, use anywhere

- A function is a **named block** of code
- It runs only when **called**
- It can receive **parameters** (inputs)
- It can **return** a result

Two declaration styles

Declaration: can be used before it is defined (hoisted)

Expression: stored in a variable

```
// Function Declaration
function greet(name) {
  return "Hello, " + name + "!";
}
console.log(greet("Ahmed"));
// Hello, Ahmed!
// Function Expression
const square = function(n) {
  return n * n;
};
console.log(square(5)); // 25
```

Functions — Arrow Functions (ES6) › A shorter, modern syntax introduced in 2015

- Introduced in **ES6 (2015)**
- Shorter syntax with `=>`
- No `function` keyword needed
- One-liners can skip `return` & braces
- Very common in modern codebases

Rule of thumb

Use arrow functions for short, simple tasks.
Use regular functions for complex logic.

```
// Regular function
function add(a, b) {
  return a + b;
}
// Same — arrow style
const add = (a, b) => a + b;
console.log(add(3, 4)); // 7
// Multi-line arrow function
const describe = (name, age) => {
  const msg =
    `${name} is ${age} years old.`;
  return msg;
};
console.log(describe("Sara", 20));
// Sara is 20 years old.
```

What is a Framework?

A framework is a **collection of pre-written code** that gives you a structure to build applications faster, without starting from zero.

React

Runs in the **browser**
(Front-end / Client-side)

Built by Meta (Facebook)
Used for building user interfaces

Component-based architecture

vs

Node.js

Runs on the **server**
(Back-end / Server-side)

Built on Chrome's V8 engine
Used for APIs & web servers

Non-blocking, event-driven

- Created by **Meta** in 2013
- Builds UIs from small reusable pieces called **components**
- Uses **JSX** — HTML-like syntax inside JavaScript
- Updates only the parts of the page that changed (**Virtual DOM**)

Used by

Facebook, Instagram, Airbnb, Netflix, WhatsApp Web, **Famous Apps**

Future & Market

```jsx
// A simple React component
function WelcomeCard({ name }) {
  return (
    <div className="card">
      <h2>Hello, {name}!</h2>
      <p>Welcome to our app.</p>
    </div>
  );
}
// Using the component
function App() {
  return <WelcomeCard name="Ahmed" />;
}
```

JSX looks like HTML but it is JavaScript under the hood.

- Created by **Ryan Dahl** in 2009
- Lets JS run on a **server**, not just a browser
- Powers REST APIs, chat apps, real-time tools
- Uses **npm** — the world's largest package registry
- Non-blocking: handles many users **simultaneously**

Used by

LinkedIn, Netflix, Uber, PayPal, NASA

```
// A simple web server with Node.js
const http = require("http");
const server = http.createServer(
  (request, response) => {
    response.writeHead(200, {
      "Content-Type": "text/plain"
    });
    response.end(
      "Hello from Node.js!"
    );
  }
);
server.listen(3000, () => {
  console.log(
    "Server running on port 3000"
  );
});
```

Conditions (`if` / `else` , `switch`)
Let your program make decisions

Loops (`for` , `while` , `for...of`)
Repeat code efficiently

Functions (declaration, expression, arrow)
Write once, reuse everywhere

Frameworks (React, Node.js)
Build real applications faster

→ What comes next

Chapter 3 will cover:

- Interacting with HTML (DOM)
- Handling events
- Common uses of JavaScript
- Advantages & limitations

CHAPTER 4

Conclusion

DOM ▪ Events ▪ Use Cases ▪ Pros & Cons

University of Bouira | April 6, 2026

```
        return "Hello, World!";
    }
```

Domains

Where JS is applied

Famous Apps

Software built with JS

Companies

Who uses JavaScript

Future

Trends & prospects

Why does this matter?

JavaScript is **everywhere** in modern technology. Understanding its real-world impact shows why it remains the **most widely used language** on the web today.

Main Application Domains › Where JavaScript is used in the real world

Web Development

Front-end **and** back-end development, dynamic interfaces, and single-page applications (SPAs).

Mobile & Desktop

Cross-platform mobile apps with **React Native** and desktop apps with **Electron**.

Server-Side

Node.js allows JS to run on servers, enabling REST APIs and real-time applications.

Other notable domains

- **Browser Games** — Canvas API & WebGL
- **Real-time apps** — Chat, live notifications
- **Cloud Functions** — Serverless computing
- **Machine Learning** — TensorFlow.js in-browser
- **IoT & Robotics** — Johnny-Five library

Did you know?

JS is the **only** language that runs natively in every web browser worldwide.

Famous Software Built with JavaScript › You use these every day

Desktop Applications

- **VS Code** — the world's most popular code editor
- **Slack** — team communication platform
- **Discord** — voice, video and text chat app
- **Atom** — open-source text editor by GitHub

Mobile Applications

- **Facebook** — partly built with React Native

Websites & Web Platforms

- **Google** — search, Maps, Gmail, Drive
- **YouTube** — video streaming & interaction
- **Twitter / X** — real-time social feed
- **Netflix** — UI built with React
- **Airbnb** — React-based front-end

Key takeaway

All these products share one thing: **JavaScript powers their user**

Companies That Rely on JavaScript › From startups to tech giants

Google

Created **V8**, the JS engine powering Chrome and Node.js. Uses JS across all its web products.

Meta (Facebook)

Created **React.js**, the most popular JS UI library, used by millions of developers.

Microsoft

Created **TypeScript** and **VS Code**. Heavily invests in the JS/Node.js ecosystem.

More major players

- **Netflix** — React-powered streaming UI
- **Airbnb** — open-sourced key React tools
- **LinkedIn** — migrated to Node.js for scale
- **PayPal** — switched to Node.js, 2× faster
- **Uber** — Node.js for real-time matching

Industry fact

98% of all websites use JavaScript on the client side. — W3Techs, 2024

Importance Today

- **#1 most used language** for 12 years in a row (Stack Overflow Survey)
- **Full-stack** language — front-end, back-end, mobile & desktop
- **2.5 million** packages on npm — largest package registry in the world
- **Essential skill** for any web or software developer today
- Powers the **real-time web** — chats, live feeds, collaborative tools

The Future of JS

- **WebAssembly** — JS working alongside near-native-speed code in browsers
- **AI integration** — TensorFlow.js and on-device ML becoming mainstream
- **Edge computing** — JS running at CDN edge nodes (Cloudflare Workers)
- **TypeScript growth** — typed JS is now the standard in large projects
- **Meta-frameworks** — Next.js, Nuxt, Astro redefining full-stack

CHAPTER 5

Group Experience

Personal Reflections ▪ Tools ▪ Lessons Learned

```
function greet() {
  return "Hello, World!";
```

University of Bouira | April 6, 2026

About this section

Each member of the group shares their **honest, personal** experience during this project — tasks completed, tools used, AI assistance, difficulties faced, and lessons learned.

Boualem
Introduction
Chapter 1

Djalil
JS Basics
Chapter 2

Ahmed
Control & Logic
Chapter 3

Anis
Conclusion
Chapter 4

Main AI tool used by the group: Claude (Anthropic)

Used for LaTeX debugging, content writing, slide design, and code explanations.

Claude

by Anthropic

claude.ai

What We Used Claude For

- **LaTeX & Beamer** — theme design, TikZ layouts, and debugging compilation errors
- **Content writing** — slide text, summaries, and explanations for a 1st-year audience
- **Code examples** — clean JS snippets for variables, loops, functions, and frameworks
- **Research filtering** — selecting the most impactful statistics and facts from large sources
- **Prompt crafting** — we learned how to write precise, effective prompts from Anthropic's founder

Why This Matters

Getting useful answers from an AI depends almost entirely on **how well you write your prompt**. We discovered a guide shared by **Dario Amodei** — CEO and co-founder of Anthropic — that taught us how to think about prompting properly.

Key Principles We Learned

- **Be specific** — vague prompts give vague answers
- **Give context** — tell the AI who you are and what you need it for
- **Set the format** — ask for bullet points, code, tables, or plain text
- **Iterate** — refine your prompt if the first

Example: Before & After

Without prompt engineering:

"Explain JavaScript."

With prompt engineering:

"Explain JavaScript in 3 bullet points for a 1st-year CS student who has never programmed. Use simple language and one short code example."

⇒ The second prompt gave us slides we could use directly.

Workflow & Lessons

- Each member owned one chapter, worked **independently** on their `.tex` files
- Shared a **common preamble** so all slides stay consistent
- One person **compiled** the final PDF
- Clear **file naming** prevents merge conflicts
- Agree on the **theme first** — saves reformatting
- AI tools speed things up, but always **verify** the output

Self-Hosted Overleaf

- Instead of paying for a subscription, we hosted our own instance using **Overleaf Community Edition** — free and open-source
- All four members could **edit simultaneously** in real time, just like the paid version
- Full **LaTeX compilation** on our own server — no package restrictions

Key Takeaway

Working as a group on a LaTeX project taught us discipline, communication, and how to merge individual work into one coherent document.

Tasks & Contribution

- Researched JavaScript history and key characteristics
- Built the **title slide**, theme colors, and `main.tex` preamble used by the whole group
- Created slides: What is JS, History, Web Trio, Popularity, Career Opportunities
- **Compiled** the final PDF and coordinated file structure for the team

General Tools

- Overleaf (LaTeX editor)

AI Tool: Claude

- *"Design a dark professional Beamer theme with blue and gold colors using TikZ."*
- *"Fix this TikZ title slide — the JS logo is not centered on the right panel."*
- *"Write content for a slide about JavaScript career opportunities."*

Difficulty → Solution

TikZ coordinate positioning was very confusing at first. I kept using `overlay` nodes that went off-screen.

Solution: Asked Claude to explain `remember picture` and `current page`

Tasks & Contribution

- Created slides covering **variables**, **data types**, **operators**, **arrays** and **objects**
- Generated code screenshots (`let` / `const` , arithmetic, comparison)
- Wrote practical JS examples suitable for first-year students with no prior experience
- Matched visual style to Boualem's theme

General Tools

- Overleaf
- Browser DevTools console (testing JS)
- Carbon now.sh (code screenshots)

AI Tool: Claude

- *"Explain the difference between `let`, `const` and `var` simply for beginners."*
- *"Show a clear example of a JS array and object for a university slide."*
- *"How do I use `tikzpicture` inside a Beamer column without overflow?"*

Difficulty → Solution

Displaying code nicely in Beamer was a challenge — `minted` required `-shell-escape` and sometimes failed on Overleaf.

Solution: Switched to code **image screenshots** styled with Carbon, then `\includegraphics`

Tasks & Contribution

- Created the **largest chapter** of the presentation (10+ slides)
- Covered: **if/else**, ternary, switch, **for/while** loops, functions, arrow functions, and JS frameworks
- Introduced **React** and **Node.js** to the audience in an accessible way
- Produced all code screenshots for Chapter 3

General Tools

- Overleaf + VS Code
- MDN Web Docs (reference)

AI Tool: Claude

- *"Explain React and Node.js in simple terms for a 1st-year CS audience."*
- *"Write a minimal but realistic Node.js server example for a presentation slide."*
- *"How do I split a long Beamer chapter into a clean overview + detail structure?"*

Difficulty → Solution

Fitting a lot of technical content (loops, functions, frameworks) into short slides without losing clarity was difficult.

Solution: Used Claude to help **summarise** each concept into 3–4 bullet points, then added a

Tasks & Contribution

- Created slides on JS **application domains**: Web, Mobile, Desktop, Server-side, IoT & ML
- Researched and presented **famous software** built with JS (VS Code, Discord, Netflix, etc.)
- Covered **major companies** using JS: Google (V8), Meta (React), Microsoft (TypeScript)
- Presented the **future of JavaScript**: WebAssembly, AI, Edge computing, TypeScript
- Designed the **Thank You / Q&A** closing slide

AI Tool: Claude

- *"List the most famous apps and software built with JavaScript for a university slide."*
- *"Explain WebAssembly and Edge computing simply for a 1st-year CS audience."*
- *"Generate a Group Experience chapter in LaTeX Beamer matching our dark blue and gold theme, one slide per member."*

Difficulty → Solution

Finding **accurate, up-to-date statistics** (npm package count, developer numbers, market share) and deciding which companies to highlight was tricky — there was too much information to fit.